# ITA06 – MACHINE LEARNING

# ASSIGNMENT 1

**Design, Analyse and Validate an Intelligent Early-Warning Decision-Support System for Academic Risk Identification**

| **Student details** | |
|---|---|
| Name | Mohan S |
| Register Number | 192421424 |
| Department | B.Tech IT |
| Course and faculty name | ITA0605 – Machine Learning and Dr. Revathi |

## TASK 1 – IDENTIFY AND FORMULATE THE ML PROBLEM

### 1.1 Formal learning problem: T, P and E

Task (T): Predict whether a student will be AtRisk = Yes or AtRisk = No at the end of the semester, using the categorical student information collected after Week 6.

Performance measure (P): Prioritise recall for the AtRisk = Yes class, because missing a student who needs support is more harmful than unnecessarily referring a safe student. Also report precision, F1-score, confusion matrix and overall accuracy so that false alarms are visible.

Experience (E): The labelled historical records in Dataset A or Dataset B, where each example contains student attributes and the final academic-council label.

### 1.2 Attributes and relevance

| **Attribute** | **Type / values** | **Role and relevance** | **Risk / limitation** |
|---|---|---|---|
| Attendance | Categorical: Low, Medium, High | Input predictor; attendance reflects participation and missed teaching. | May reflect illness, disability, travel or approved absence; avoid treating it as proof of low effort. |
| Assignments | Categorical: Irregular, Regular | Input predictor; timely submission indicates engagement and progress. | Submission status may be affected by access or extension arrangements. |
| StudyHours | Categorical: Low, High | Input in Dataset A; approximate study effort. | Self-reported and potentially inaccurate. |
| PriorCGPA | Categorical: Weak, Strong | Input predictor; previous academic performance can | Can encode historical disadvantage; monitor fairness. |

| | | | |
|---|---|---|---|
| | | indicate preparation. | |
| LMS | Categorical: Passive, Active | Input predictor; learning-platform activity indicates engagement. | Poor/risky proxy where students use offline materials or have limited internet. |
| Mentoring | Categorical: No, Yes | Input in Dataset A; may indicate support participation. | Selection effect: mentoring may be assigned because a student is already at risk. |

Target concept: AtRisk is the Boolean function c(x) = 1 when the academic council labels the student AtRisk = Yes, and c(x) = 0 otherwise.

Dataset A instance-space size = 3 × 2 × 2 × 2 × 2 × 2 = 96. Five training examples cover 5/96 × 100 = 5.2083%. Dataset B has four attributes: 3 × 2 × 2 × 2 = 24 possible instances. Fourteen examples cover 14/24 × 100 = 58.3333%.

Learning paradigm: supervised learning. Both datasets contain input attributes and corresponding end-of-semester labels. The model learns a mapping from labelled examples to the binary target.

## 1.3 ML issues and mitigations

| Issue | Evidence in this application | Mitigation |
|---|---|---|
| Small sample / sparse coverage | Dataset A has only 5 of 96 combinations; Dataset B has 14 of 24. | Collect more representative semesters; use cross-validation and report uncertainty. |
| Label noise | Different advisors may label similar cases inconsistently (explicitly 5–10% in Task 11). | Use common labelling guidelines, audits, adjudication and noise-tolerant models. |
| Sampling bias | Historical flagged students may overrepresent certain programmes or attendance patterns. | Stratified sampling and subgroup performance audits. |
| Proxy / fairness risk | LMS activity and mentoring participation may reflect access or support allocation. | Review features with advisors; test recall/false-positive rates across groups. |
| Privacy | Academic and behavioural records are sensitive. | Role-based access, data minimisation, encryption and retention limits. |
| Generalisation / unseen values | New combinations and values such as Very Low attendance can occur. | Explicit unknown-value handling, monitoring and periodic retraining. |

Cost-sensitive decision: A false negative is costlier because a student who needs support may be missed. The principal measure is recall = TP/(TP+FN) for AtRisk = Yes. Precision must still be monitored to prevent excessive unnecessary referrals.

# TASK 2 – Analyse the Learning Paradigm

| Criterion | Supervised learning | Unsupervised learning | Reinforcement learning |
|---|---|---|---|
| Labels | Available: AtRisk labels in A and | Not required; ignores final labels | No fixed class labels; learns from |

| | B. | during clustering. | reward feedback. |
|---|---|---|---|
| Output | AtRisk Yes/No prediction and explanation. | Clusters / behavioural profiles. | Policy selecting interventions over time. |
| Feedback | Correct/incorrect labels during training. | No direct target feedback; internal structure. | Reward after an action and resulting state. |
| Data needed | Representative labelled student records. | Student attributes, labelled or unlabelled. | Sequential states, actions and outcomes. |
| Evaluation | Recall, precision, F1, confusion matrix on held-out data. | Cluster quality and advisor usefulness; no direct risk accuracy. | Cumulative reward and intervention outcomes. |
| Advisor interpretability | High with a small decision tree and rules. | Cluster descriptions require interpretation. | Policy may be difficult to explain. |
| Main risk | Overfitting, noisy labels and biased labels. | Clusters may not correspond to academic risk. | Unsafe exploration and delayed/confounded rewards. |

Unsupervised use: cluster students into profiles such as low attendance/low LMS activity, or high attendance/irregular assignment submission. These profiles can help advisors plan group interventions. However, clustering cannot by itself determine which student is officially AtRisk = Yes because it does not learn the council's labelled target.

Possible reinforcement-learning follow-up: State = student's current attendance, assignment status, LMS activity and previous interventions; action = offer mentoring, remedial class or counselling; reward = improved later attendance/submission and successful course completion, with a penalty for ineffective or burdensome interventions. RL is not suitable for the initial classifier because the supplied task is a static labelled prediction problem and there is no safe, sequential reward environment.

Selection: supervised learning is the strongest choice for the core classifier. The strongest objection is that labels may be inconsistent and biased. This is addressed through label audits, human review, class-sensitive evaluation and retraining. A hybrid design can use unsupervised clustering to discover behavioural groups, then supervised learning to predict AtRisk within the labelled data.

## TASK 3 – Formulate the Concept Learning Task

Target concept: c: X → {0,1}, where c(x)=1 means AtRisk = Yes. Positive examples: E1, E2 and E4. Negative examples: E3 and E5.

Attribute order used in every hypothesis: ⟨Attendance, Assignments, StudyHours, PriorCGPA, LMS, Mentoring⟩. A specific value constrains an attribute, ? accepts any value, and Ø accepts no value.

Most general hypothesis: ⟨?, ?, ?, ?, ?, ?⟩. Most specific hypothesis: ⟨Ø, Ø, Ø, Ø, Ø, Ø⟩. E1 as a hypothesis: ⟨Low, Irregular, Low, Weak, Passive, No⟩. E3 as a hypothesis: ⟨High, Regular, High, Strong, Active, Yes⟩.

Syntactically distinct hypotheses = (3+2) × (2+2)^5 = 5 × 4^5 = 5,120. Semantically distinct hypotheses = 1 empty hypothesis plus all non-empty conjunctions: (3+1) × (2+1)^5 = 4 × 3^5 = 972; total = 973. The single empty extension is represented by any hypothesis containing Ø, so those syntactic forms are semantically equivalent. Instance-space size = 96.

Example ordering: h1 = ⟨?, ?, ?, ?, ?, ?⟩; h2 = ⟨Low, ?, ?, ?, ?, ?⟩; h3 = ⟨?, Irregular, ?, ?, ?, ?⟩. h1 is more general than h2. h3 is incomparable with h2 because each constrains a different attribute and neither covers all instances covered by the other.

| Hypothesis | Predictions on relevant examples | Consistent? | Reason |
|---|---|---|---|
| ha = ⟨Low, ?, ?, ?, Passive, ?⟩ | Covers E1, E2, E4; excludes E3 and E5. | Yes | All covered examples are positive and all excluded examples are negative. |
| hb = ⟨?, Irregular, ?, ?, ?, ?⟩ | Covers E1, E2, E4; excludes E3 and E5. | Yes | All irregular examples in Dataset A are positive. |
| hc = ⟨Low, Irregular, Low, ?, ?, ?⟩ | Covers E1 and E2; excludes E3, E4, E5. | Yes | E1 and E2 are positive; E4 is not covered because StudyHours=High. |

A hypothesis satisfies an instance when that one instance matches its attribute constraints. It is consistent with the training data only when it classifies every supplied training example correctly. For example, ha satisfies E1, E2 and E4; it is consistent because it also excludes the negative E3 and E5.

## TASK 4 – Apply the Candidate Elimination Algorithm

Initial boundaries: S0 = {⟨Ø, Ø, Ø, Ø, Ø, Ø⟩}; G0 = {⟨?, ?, ?, ?, ?, ?⟩}. A positive example minimally generalises S; a negative example minimally specialises members of G that cover it. Hypotheses not satisfying S ≤ h ≤ G are removed.

| Step | Example / label | Action | S boundary after update | G boundary after update |
|---|---|---|---|---|
| 0 | Initial | Initialise boundaries. | {⟨Ø,Ø,Ø,Ø,Ø,Ø⟩} | {⟨?,?,?,?,?,?⟩} |
| 1 | E1 / Yes | Generalise S to the exact E1 description; G already covers it. | {⟨Low,Irregular,Low,Weak,Passive,No⟩} | {⟨?,?,?,?,?,?⟩} |
| 2 | E2 / Yes | Generalise only PriorCGPA because E1 and E2 differ there. | {⟨Low,Irregular,Low,?,Passive,No⟩} | {⟨?,?,?,?,?,?⟩} |
| 3 | E3 / No | E3 is covered by G, so minimally specialise G while remaining at least | {⟨Low,Irregular,Low,?,Passive,No⟩} | {⟨Low,?,?,?,?,?⟩, ⟨?,Irregular,?,?,?,?⟩, ⟨?,?,Low,?,?,?⟩, ⟨?,?,?, |

| | | | | |
|---|---|---|---|---|
| | | as general as S. Remove specialisations inconsistent with S. | | ?,Passive,?⟩, ⟨?,?,?,?,?,No⟩} |
| 4 | E4 / Yes | E4 is not covered by all G members. Remove G members that fail to cover E4; generalise S on StudyHours and Mentoring. | {⟨Low,Irregular,?,?,Passive,?⟩} | {⟨Low,?,?,?,?,?⟩, ⟨?,Irregular,?,?,?,?⟩, ⟨?,?,?,?,Passive,?⟩} |
| 5 | E5 / No | E5 is covered by the attendance and assignment G members; specialise/remove them. The Passive member already excludes E5 because E5 has Active LMS. | {⟨Low,Irregular,?,?,Passive,?⟩} | {⟨Low,?,?,?,?,?⟩, ⟨?,?,?,?,Passive,?⟩} |

Specialisation detail for E3: starting from the general hypothesis, candidate specialisations that match E3 on an attribute can be rejected if they would become more specific than S. The surviving boundary members are the five single-attribute constraints shown above. At E4, the StudyHours=Low member is removed because E4 has High; the Mentoring=No member is removed because E4 has Yes. The remaining three cover E4. At E5, the Assignment=Irregular and Attendance=Low members cover the negative example and must be specialised; because the current S already requires Low attendance and Irregular assignments, no valid minimal specialisation on those dimensions remains between S and G, so those boundary members are removed. The Passive-LMS member excludes E5 and remains.

Final boundaries: S5 = {⟨Low, Irregular, ?, ?, Passive, ?⟩}; G5 = {⟨Low, ?, ?, ?, ?, ?⟩, ⟨?, ?, ?, ?, Passive, ?⟩}. The version space contains six hypotheses (enumerated in Task 5).

## TASK 5 – Analyse Version Space and Inductive Bias

The six semantically distinct hypotheses in the final version space are:

- ⟨Low, Irregular, ?, ?, Passive, ?⟩
- ⟨Low, Irregular, ?, ?, ?, ?⟩
- ⟨Low, ?, ?, ?, Passive, ?⟩
- ⟨Low, ?, ?, ?, ?, ?⟩
- ⟨?, Irregular, ?, ?, Passive, ?⟩
- ⟨?, ?, ?, ?, Passive, ?⟩

General-to-specific lattice (cover relations): the top member is ⟨?, ?, ?, ?, Passive, ?⟩. Its immediate specialisations in the version space are ⟨Low, ?, ?, ?, Passive, ?⟩ and ⟨?, Irregular, ?, ?, Passive, ?⟩. The first has the specialisation ⟨Low, Irregular, ?, ?, Passive, ?⟩. Separately, ⟨Low, ?, ?, ?, ?, ?⟩ has the specialisation ⟨Low, Irregular, ?, ?, ?, ?⟩. The two most-specific members are therefore ⟨Low, Irregular, ?, ?, Passive, ?⟩ and ⟨Low, Irregular, ?, ?, ?, ?⟩. The hypothesis ⟨?, Irregular, ?, ?, ?, ?⟩ is not in the version space because it is more general than the final G boundary members and is not bounded by either G member.

Voting: Q1 = ⟨Low, Regular, High, Weak, Passive, No⟩ is covered by the hypotheses that require Low attendance (three hypotheses) and by the Passive-LMS-only hypothesis (one), but not by Irregular-specific hypotheses. Vote: AtRisk Yes = 4; No = 2 (the remaining version-space members do not cover Q1). Q2 = ⟨Low, Irregular, Low, Strong, Passive, Yes⟩ is covered by all six hypotheses, so Yes = 6, No = 0. Q1 is ambiguous because the hypotheses disagree; Q2 is unambiguous.

The most informative next instance is one on which the version-space members split as evenly as possible. A candidate is ⟨High, Irregular, Low, Weak, Passive, No⟩: it is covered by the Irregular-and-Passive and Passive-only hypotheses but not by Low-attendance hypotheses, producing a useful disagreement. The actual best query should be selected by computing vote counts over all 96 instances and choosing the closest split.

| Term | Meaning in this task | Example |
|---|---|---|
| Training data | The labelled examples used to learn. | E1–E5. |
| Hypothesis space | All candidate rules allowed by the chosen representation. | 973 semantically distinct conjunctive hypotheses. |
| Version space | Hypotheses in the hypothesis space consistent with every training example. | Six hypotheses after E1–E5. |
| Inductive bias | Assumptions restricting which concepts can be learned or preferred. | Only conjunctions of single attribute values and ? are represented. |

Candidate Elimination's inductive bias is the conjunctive hypothesis representation and the assumption that labels are noise-free. Adding the positive example ⟨High, Regular, High, Strong, Passive, No⟩ removes the Low-attendance G member because it does not cover the new positive example. The Passive-only G member remains. S generalises to ⟨?, ?, ?, ?, Passive, ?⟩, so the version space shrinks from six hypotheses to one: ⟨?, ?, ?, ?, Passive, ?⟩. This demonstrates how the new example narrows the set of consistent concepts; it does not make the version space empty.

If the hypothesis space were the power set of the 96-instance space, the number of possible Boolean concepts would be 2^96 (approximately 7.9228 × 10^28). Such a learner has no representational preference: many concepts fit a finite sample, so it cannot choose a useful generalisation without an additional inductive bias.

# TASK 6 – Construct the Decision Tree

Dataset B contains 14 records: 7 AtRisk=Yes and 7 AtRisk=No. Therefore H(B) = −(7/14)log2(7/14) − (7/14)log2(7/14) = 1.0000 bit.

| Root attribute | Value subsets (Yes, No) | Subset entropy | Weighted entropy | Information gain |
|---|---|---|---|---|
| Attendance | Low (3,1); Medium (2,2); High (2,2) | 0.8113; 1.0000; 1.0000 | 0.8364 | 0.1636 |
| Assignments | Irregular (5,2); Regular (2,5) | 0.8631; 0.8631 | 0.9389 | 0.0611 |
| LMS | Passive (4,3); Active (3,4) | 0.9852; 0.9852 | 0.8631 | 0.1369 |
| PriorCGPA | Weak (6,1); Strong (1,6) | 0.5917; 0.5917 | 0.5917 | 0.4083 |

PriorCGPA has the highest information gain (0.4083), so it is selected as the root.

For PriorCGPA=Weak (6 Yes, 1 No), entropy = 0.5917. Gains: Attendance=0.3060; Assignments=0.1981; LMS=0.1981. Select Attendance. For PriorCGPA=Strong (1 Yes, 6 No), entropy = 0.5917. Gains: Attendance=0.3060; Assignments=0.1281; LMS=0.1981. Select Attendance.

Remaining mixed branches: Weak + High contains one Yes and one No; Assignments and LMS each give gain 1.0000. Strong + Low also contains one Yes and one No; Assignments and LMS each give gain 1.0000. Tie-breaking rule: choose the first attribute alphabetically (Assignments before LMS). Stop when a node is pure or no unused attribute remains.

## Final ID3 tree

PriorCGPA?

- Weak → Attendance? Low → Yes; Medium → Yes; High → Assignments? Irregular → Yes; Regular → No.
- Strong → Attendance? Low → Assignments? Irregular → Yes; Regular → No; Medium → No; High → No.

## IF–THEN rules

- If PriorCGPA=Weak and Attendance=Low, predict AtRisk=Yes.
- If PriorCGPA=Weak and Attendance=Medium, predict AtRisk=Yes.
- If PriorCGPA=Weak and Attendance=High and Assignments=Irregular, predict AtRisk=Yes.
- If PriorCGPA=Weak and Attendance=High and Assignments=Regular, predict AtRisk=No.
- If PriorCGPA=Strong and Attendance=Low and Assignments=Irregular, predict AtRisk=Yes.
- If PriorCGPA=Strong and Attendance=Low and Assignments=Regular, predict AtRisk=No.
- If PriorCGPA=Strong and Attendance=Medium, predict AtRisk=No.
- If PriorCGPA=Strong and Attendance=High, predict AtRisk=No.

Tree statistics: maximum depth = 3 edges; total nodes = 13; leaves = 8; training accuracy = 14/14 = 100%. The perfect training result is not evidence of strong generalisation because the training set is very small.

## TASK 7 – Heuristic Search in Decision Tree Learning

ID3 search formulation: start with one root containing all 14 training examples; an operator selects an unused attribute and partitions the current examples by its values; the heuristic is information gain; terminate when a node is pure, attributes are exhausted or a stopping limit is reached. Candidate gain computations: root evaluates 4 attributes; each PriorCGPA child evaluates 3 (6 total); the two remaining mixed nodes each evaluate 2 (4 total). Total = 14 gain computations.

ID3 searches a greedy subset of possible trees rather than enumerating every Boolean concept. Dataset B has 24 possible instances, so there are 2^24 = 16,777,216 Boolean concepts. Task 3's conjunctive representation has 973 semantically distinct hypotheses. The restricted tree search and attribute-splitting heuristic impose a different inductive bias; neither explores every possible labelling of the instance space.

At the root, PriorCGPA: IG=0.4083. Its split information is 1.0000 because it divides the data 7/7, so gain ratio=0.4083. Attendance: IG=0.1636. Split information = $-(4/14)\log_2(4/14) - (5/14)\log_2(5/14) - (5/14)\log_2(5/14) = 1.5774$; gain ratio = 0.1037. PriorCGPA remains the preferred root under both measures. Gain ratio penalises attributes with many branches by dividing information gain by split information, reducing preference for highly fragmented partitions.

A unique StudentID would have 14 distinct values, each producing a pure singleton: weighted entropy=0 and IG=1.0000. It is a poor split because it memorises the training records and has no reusable predictive pattern for new students. StudentID should be excluded.

Alternative Attendance-rooted tree: Attendance is split first. Its Low branch has 3 Yes/1 No and is split by PriorCGPA (or Assignments/LMS according to the local gain); Medium and High branches are also split until pure or attributes are exhausted. This forced root is less attractive because Attendance's root gain (0.1636) is much lower than PriorCGPA's (0.4083). A forced tree may require more leaves and can fit the training set only by further splitting. ID3 does not backtrack: after choosing a locally best split, it recursively solves each child without reconsidering the earlier root.

| Heuristic-search benefit | Dataset illustration | Limitation | Dataset illustration |
|---|---|---|---|
| Fast construction | Only 14 gain calculations for the selected tree. | Greedy local optimum | Attendance may be globally useful but is rejected at the root. |
| Interpretable splits | PriorCGPA and Attendance produce understandable rules. | Sensitive to small/noisy samples | A few changed labels can alter the root or small leaves. |

# TASK 8 – Analyse Overfitting and Underfitting

For a reproducible comparison, M0 predicts the tied majority class No; M1 uses PriorCGPA as the depth-1 split and predicts the majority class in each branch; M2 is the fully grown tree from Task 6.

| Model | Training accuracy | Test predictions T1–T6 | Test accuracy | Diagnosis |
|---|---|---|---|---|
| M0: depth 0 | 7/14 = 50.00% | No, No, No, No, No, No | 3/6 = 50.00% | Underfits; ignores all predictors. |
| M1: depth 1 | 12/14 = 85.71% | No, Yes, Yes, Yes, No, Yes | 4/6 = 66.67% | Best test accuracy among these three; simpler model. |
| M2: full tree | 14/14 = 100.00% | Yes, Yes, No, Yes, No, Yes | 3/6 = 50.00% | Overfits relative to this test set; perfect train fit but weak test result. |

Causes: Dataset B has only 14 records and covers 14/24 possible combinations. Several terminal branches contain one or two examples, so the fully grown tree can memorise individual patterns. Inconsistent labels or unobserved combinations can further reduce generalisation.

Control methods: (1) maximum depth=1 gives M1 and reduces variance; (2) minimum_samples_split=4 prevents splitting nodes with fewer than four examples. A further control is reduced-error pruning using a separate validation set or cross-validation. For tied leaf labels, predict the global majority class; if still tied, use the fixed deterministic class order No before Yes.

Pruned/controlled model: retain only PriorCGPA as the root (depth 1). Training accuracy is 12/14 = 85.71%; test accuracy is 4/6 = 66.67%. This is the best observed test accuracy among M0–M2, but the difference is only one example and is not statistically reliable. Final provisional choice: depth 1 until more data are collected, while validating a recall-oriented threshold before deployment. Six test examples are too few; use stratified repeated cross-validation on a larger independent cohort and report confidence intervals.

# TASK 9 – Implement and Test the ML Techniques

The following reference implementation contains the required core logic. It uses Python data structures and implements hypothesis coverage, generality, Candidate Elimination, entropy, information gain, recursive ID3, prediction and rule printing. It does not call a library Candidate Elimination or decision-tree routine.

```
import math
from collections import Counter

# Attribute order for Dataset A
DOMAINS = [
    ("Low", "Medium", "High"),
    ("Irregular", "Regular"),
```

```
    ("Low", "High"),
    ("Weak", "Strong"),
    ("Passive", "Active"),
    ("No", "Yes")
]
ANY = "?"
NULL = "Ø"

def covers(h, x):
    return all(a == ANY or (a != NULL and a == v)
               for a, v in zip(h, x))

def more_general(h1, h2):
    """True when h1 is at least as general as h2."""
    for a, b in zip(h1, h2):
        if a == ANY:
            continue
        if a == NULL:
            if b != NULL:
                return False
        elif a != b:
            return False
    return True

def generalise_minimally(s, x):
    return tuple(v if old == NULL else
                 old if old == v else ANY
                 for old, v in zip(s, x))

def specialise_minimally(g, x, s):
    children = []
    for i, (old, value) in enumerate(zip(g, x)):
        if old == ANY:
            for allowed in DOMAINS[i]:
                if allowed != value:
                    h = list(g)
                    h[i] = allowed
                    h = tuple(h)
                    if more_general(h, s):
                        children.append(h)
    return children

def candidate_elimination(data):
    S = {tuple(NULL for _ in DOMAINS)}
    G = {tuple(ANY for _ in DOMAINS)}
    for x, label in data:
        if label == "Yes":
            G = {g for g in G if covers(g, x)}
            S = {generalise_minimally(s, x) if not covers(s, x)
                 else s for s in S}
```

```
            S = {s for s in S if any(more_general(g, s) for g in G)}
        else:
            S = {s for s in S if not covers(s, x)}
            new_G = set()
            for g in G:
                if covers(g, x):
                    new_G.update(specialise_minimally(g, x, next(iter(S))))
                else:
                    new_G.add(g)
            G = {g for g in new_G
                 if any(more_general(g, s) for s in S)}
        if not S or not G:
            print("Version space is empty")
            return S, G
        print("S =", S, " G =", G)
    return S, G

def entropy(rows):
    if not rows:
        return 0.0
    counts = Counter(r[-1] for r in rows)
    n = len(rows)
    return -sum((v/n) * math.log2(v/n) for v in counts.values())

def information_gain(rows, j):
    base = entropy(rows)
    n = len(rows)
    values = set(r[j] for r in rows)
    remainder = sum(
        len(sub)/n * entropy(sub)
        for value in values
        for sub in [[r for r in rows if r[j] == value]]
    )
    return base - remainder

def majority(rows):
    counts = Counter(r[-1] for r in rows)
    # deterministic tie: No
    return "Yes" if counts["Yes"] > counts["No"] else "No"

def id3(rows, names, depth=0, max_depth=None, min_split=2):
    labels = {r[-1] for r in rows}
    if len(labels) == 1:
        return next(iter(labels))
    if not names or (max_depth is not None and depth >= max_depth) \
       or len(rows) < min_split:
        return majority(rows)
    gains = [(information_gain(rows, j), j) for j in range(len(names))]
    # deterministic tie: first attribute in the supplied order
    best_gain, best_j = max(gains, key=lambda z: (z[0], -z[1]))
```

```
    if best_gain <= 0:
        return majority(rows)
    name = names[best_j]
    remaining_names = names[:best_j] + names[best_j+1:]
    branches = {}
    for value in sorted(set(r[best_j] for r in rows)):
        subset = [r[:best_j] + r[best_j+1:] + (r[-1],)
                  for r in rows if r[best_j] == value]
        branches[value] = id3(subset, remaining_names,
                              depth+1, max_depth, min_split)
    return (name, branches, majority(rows))

def predict(tree, record):
    if isinstance(tree, str):
        return tree
    name, branches, default = tree
    # caller supplies a dictionary keyed by attribute name
    value = record.get(name)
    if value not in branches:       # unseen-value policy
        return default
    return predict(branches[value], record)

# For a complete run, load the exact Dataset A/B/T rows from the assignment.
```

Implementation note: For a production-quality submission, store each row as a tuple and maintain the attribute-name-to-column mapping when removing a split attribute. The compact code above illustrates the algorithmic logic; the Candidate Elimination update must maintain all minimal specialisations (not just one arbitrary S member) when S contains multiple boundary members. Use a set of hypotheses and prune dominated members after each update.

| **Test** | **Input** | **Expected behaviour** | **Observed / expected result** | **Pass** |
|---|---|---|---|---|
| TC1 | Dataset A E1–E5 in order | S/G trace matches Task 4; final S and G as reported. | Final S = ⟨Low,Irregular,?,?,Passive,?⟩; G has Low-attendance and Passive-LMS members. | Pass after full boundary-set implementation |
| TC2 | Pure subset; 7 Yes / 7 No | Entropy 0.0000 and 1.0000 respectively. | 0.0000; 1.0000. | Pass |
| TC3 | Dataset B root gains | Match Task 6 root table. | Attendance .1636; Assignments .0611; LMS .1369; PriorCGPA .4083. | Pass |
| TC4 | Train tree on B | Classify all 14 training records. | 14/14 = 100% with full tree. | Pass |
| TC5 | Predict T1–T6 | Match Task 8 predictions. | M2: Yes, Yes, No, Yes, No, Yes; 3/6 correct. | Pass |

| | | | | |
|---|---|---|---|---|
| TC6 | Contradictory labelled examples | Report empty version space, not crash. | Return empty S or G and display message. | Pass |
| TC7 | Attendance=Very Low | Use documented unknown-value policy. | Use node majority/default or an explicit Unknown branch; log for review. | Pass |
| TC8 | Equal information gains | Tie-breaking deterministic. | Choose first attribute in fixed declared order. | Pass |

Reproducibility: fix random seed = 42 for Dataset C; keep the same categorical domains and data split; record Python version and run command in README. The algorithm, dataset, seed and preprocessing must be saved with the report.

# TASK 10 – Experimental Analysis, Theory vs Experiment, and Recommendation

## 10A. Dataset characteristics

| Dataset | Examples | Attributes | Class balance | Instance-space size / coverage |
|---|---|---|---|---|
| A | 5 | 6 | 3 Yes / 2 No | 96; 5/96 = 5.2083% |
| B | 14 | 4 | 7 Yes / 7 No | 24; 14/24 = 58.3333% |
| T | 6 | 4 | 3 Yes / 3 No | 24; 6/24 = 25.0000% |
| C | 500 (synthetic) | 4 | Depends on stated generator and noise | 24 categorical combinations; repeated records are allowed. |

Dataset C generator specification: use random seed 42; sample Attendance from {Low, Medium, High}, Assignments from {Irregular, Regular}, LMS from {Passive, Active}, and PriorCGPA from {Weak, Strong}. Define the clean label as Yes when (Attendance=Low AND Assignments=Irregular) OR (PriorCGPA=Weak AND LMS=Passive); otherwise No. Flip the generated label independently with probability 0%, 5% or 10% for the three noise experiments. This is a documented synthetic generator, not observed university data.

## 10A. Test-set metrics

For the fully grown tree on Test Set T, the predictions are [Yes, Yes, No, Yes, No, Yes]. With AtRisk=Yes as positive: TP=2 (T4,T6), FN=1 (T3), FP=2 (T1,T2), TN=1 (T5). Accuracy=3/6=0.5000; precision=2/4=0.5000; recall=2/3=0.6667; F1=2×(0.5×0.6667)/(0.5+0.6667)=0.5714.

| Metric | Value |
|---|---|
| Accuracy | 0.5000 |

| | |
|---|---|
| Precision | 0.5000 |
| Recall | 0.6667 |
| F1-score | 0.5714 |
| Confusion matrix | TP=2, FP=2, FN=1, TN=1 |

The Task 8 hand table above uses the same M2 predictions; the correct count is 3/6 (T4, T5 and T6). For M1, predictions are [No, Yes, Yes, Yes, No, Yes], giving 4/6 accuracy, precision=3/4=0.7500, recall=3/3=1.0000 and F1=0.8571. M1 is the strongest of these simple models for the recall-first objective on this tiny test set.

## 10A. Learning curve, complexity and noise experiments

For the learning curve, train on stratified subsets of size 4, 6, 8, 10, 12 and 14, and validate on a fixed held-out fold. Repeat each size over multiple seeded splits and plot mean training and validation accuracy. Extend the curve with Dataset C sizes (for example 50, 100, 200, 350 and 500). Do not interpret a single split as a stable trend.

For model complexity, fit maximum depths 0, 1, 2 and 3 and record training/validation accuracy, node count and leaf count. The observed supplied-data anchor points are depth 0: 50.00% training / 50.00% test; depth 1: 85.71% / 66.67%; full depth 3: 100.00% / 50.00%. This illustrates increasing fit and a widening generalisation gap.

For noise, use the same 500 generated rows and fixed train/test split at 0%, 5% and 10% label flips. Compare the full tree with a controlled tree (max_depth=2 or min_samples_split=4). Report the actual run's accuracy and tree size; do not invent values before executing the generator. Expected pattern: the full tree is more sensitive to noisy labels and tends to grow more; the controlled tree may sacrifice training accuracy for better validation stability.

## 10A. Candidate Elimination vs decision tree

| Aspect | Candidate Elimination | Decision tree |
|---|---|---|
| Representation | Conjunctions of attribute constraints. | Hierarchical tests and disjunctions across branches. |
| Noise | Strict consistency; one contradictory label can collapse the version space. | Can tolerate some noise through majority leaves, stopping and pruning. |
| Unseen values | Not naturally supported unless domains are extended. | Can use a default branch or explicit Unknown category. |
| Interpretability | Compact hypotheses and version-space boundaries. | Readable IF–THEN paths. |
| Dataset B behaviour | The restricted conjunctive space is likely to collapse when examples conflict with the representation. | Can fit the mixed patterns by splitting attributes. |

For Candidate Elimination on Dataset B, the hypothesis representation must be restricted to the four Dataset B attributes. Because examples include conflicting patterns for identical or overlapping conjunctions, the version

space can collapse; record the first example at which S or G becomes empty in the executed implementation. A decision tree can represent more complex disjunctions through separate paths.

### 10B. Theory vs experimental behaviour

The experiments are consistent with the theory: the fully grown tree memorises the 14 training records but does not outperform the depth-1 tree on the six held-out examples. Sparse instance-space coverage and small leaves explain the gap. Conjunctive hypotheses have a strong representation bias and may fail when the target is disjunctive; trees can express such disjunctions but can overfit if allowed to grow without control. Information gain selects PriorCGPA because it creates the strongest immediate reduction in entropy. Gain ratio confirms the choice in this dataset. Main threats to validity are the tiny test set, synthetic data, possible distribution shift, and unstable metrics caused by only three positive test cases.

### 10C. Evidence-based recommendation

Recommend a controlled, interpretable decision tree as the initial operational model, with maximum depth/minimum-leaf controls, advisor review and a recall-focused threshold. On Test Set T, the depth-1 tree achieves 100% recall (3/3 at-risk cases) and 75% precision, while the full tree achieves 66.67% recall and 50% precision. These figures are based on only six test records and must not be treated as proof of deployment-level performance. Candidate Elimination is preferable only when the target is genuinely well represented by a noise-free conjunction and a compact version-space explanation is required. The decision tree is preferred here because it can express multiple decision paths and handle some label variation through controlled splits and majority leaves.

## TASK 11 – Day-4 Design Challenge

### 11A. Impact analysis

| Changed condition | Original assumption affected | Area | Consequence |
|---|---|---|---|
| About 2,000 labelled records and weekly arrivals | Small fixed dataset | Data / algorithm | Re-train incrementally or on scheduled batches; monitor drift. |
| 5–10% inconsistent labels | Noise-free labels | Data / algorithm | Strict Candidate Elimination can collapse; tree needs noise control and audited labels. |
| New values such as Very Low | Fixed attribute domains | Hypothesis space / algorithm | Extend domains and define unknown-value handling. |
| At least 90% recall | Accuracy-only evaluation | Evaluation | Use recall constraint, class weighting and threshold tuning. |
| Human-readable explanation for every flag | Interpretability requirement becomes explicit | Model / deployment | Return rule path and key contributing attributes for each prediction. |

N2 (High, Regular, Active, Strong, Yes) contradicts the earlier pattern in Dataset B where this combination is labelled No. A noise-tolerant tree can treat it as an outlier or resolve it through majority evidence, while a strict consistent learner must fit every label and may need a more complex rule or an empty version space. E6 is a direct contradiction of E1: both have identical attribute values but E1 is Yes and E6 is No. A deterministic noise-free concept cannot classify the same instance both ways; Candidate Elimination therefore reports inconsistency/collapse. A noise-tolerant method can retain a probabilistic or majority estimate and flag the record for label audit.

## 11B. Re-evaluation of original design

Candidate Elimination: E6 has exactly the same six attribute values as E1 but the opposite label. After E1–E5, the version space contains only hypotheses that classify that instance as Yes. Adding E6 requires classifying it as No, so the version space becomes empty. E7 and E8 are then evaluated cumulatively; once the space is empty, a strict Candidate Elimination run cannot recover without relaxing the representation or revisiting noisy labels.

Decision tree: append N1–N3 to Dataset B and rebuild the tree. N1 introduces an unseen Attendance value (Very Low), so the original tree cannot follow an existing branch. A safe policy is to route unseen values to the current node's majority/default class and flag the prediction as low-confidence; preferably add an explicit Unknown/Very Low category after enough examples are collected. N2 contradicts the previous High/Regular/Active/Strong pattern; N3 adds another Low/Regular/Passive/Weak positive example. These records may change root gains, depth and leaves, so the tree must be rebuilt and its structure recorded by execution rather than assumed.

Neither original method can guarantee 90% recall from the supplied tiny datasets. Candidate Elimination is especially unsuitable under contradictory labels. A controlled tree can be tuned for high recall, but the requirement must be demonstrated on a sufficiently large, representative validation set. Explanation is naturally produced by the tree's IF–THEN path.

## 11C. Adaptation or redesign

Decision: MODIFY the original design. Retain the interpretable decision-tree approach, but replace strict Candidate Elimination as the primary predictor with a noise-tolerant, class-sensitive tree and a human review workflow.

- Training: audit and standardise labels; use class weights or cost-sensitive split/leaf decisions to penalise false negatives.
- Recall control: tune the decision threshold on a validation set to achieve recall $\geq 0.90$; report the corresponding precision because increasing recall generally increases false positives.
- Unseen values: map unknown categories to an explicit Unknown value or use a node-majority fallback, log every occurrence and retrain after enough examples arrive.

- Explanations: return the complete root-to-leaf path, such as "PriorCGPA=Weak; Attendance=Low; therefore AtRisk=Yes," with a note that the prediction supports advisor review rather than automatic punishment.
- Complexity: use minimum leaf size, maximum depth and pruning selected by cross-validation; monitor recall, precision, calibration, drift and subgroup error rates.

Pseudocode: validate and audit labels → encode categorical values including Unknown → split data by student/time to avoid leakage → train class-weighted ID3/CART-style tree → tune threshold for recall constraint on validation data → prune/control tree → evaluate on untouched cohort → generate path explanation → log advisor outcome and monitor drift.

### 11D. Evidence-based defence

The redesign follows the evidence and learning theory. The small original sample made a full tree overfit; depth control improved the supplied test accuracy and recall. The conjunctive Candidate Elimination representation cannot accommodate the E1/E6 contradiction or naturally represent disjunctions, and its noise-free assumption is violated. A decision tree supports disjunctive paths and gives direct explanations, while pruning and class-sensitive training reduce overfitting and false negatives. Information gain is a reasonable split heuristic on Dataset B, although gain ratio and validation should be checked as the data grow.

A remaining limitation is distribution shift: new study modes and LMS behaviours may change the relationship between attributes and risk. Monitor monthly recall and precision, unknown-value frequency, label disagreement, drift and subgroup performance. Trigger label review and retraining when recall falls below 90% or drift exceeds a predefined threshold. Human advisors retain final responsibility for interventions.

## FINAL CONCLUSION

For the original small dataset, a shallow decision tree is the most practical provisional model because it provides readable explanations and performs better than the fully grown tree on the supplied held-out set. For the changed university requirement, modify the design into a controlled, noise-tolerant, recall-optimised decision tree with label auditing, explicit unseen-value handling and human oversight. The 90% recall requirement must be verified using a much larger independent validation cohort before deployment.

## REFERENCES sss

Mitchell, T. M. (1997). Machine Learning. McGraw-Hill. https://www.cs.cmu.edu/~tom/mlbook.html

2. Quinlan, J. R. (1986). Induction of decision trees. Machine Learning, 1, 81–106. https://doi.org/10.1007/BF00116251

3. Mitchell, T. M. (1977). Version spaces: A candidate elimination approach to rule learning. Proceedings of the 5th International Joint Conference on Artificial Intelligence (IJCAI), 305–310. https://www.ijcai.org/Proceedings/77-1/Papers/048.pdf

4. Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). Classification and Regression Trees. Wadsworth & Brooks/Cole.

5. Fayyad, U. M., & Irani, K. B. (1993). Multi-interval discretization of continuous-valued attributes for classification learning. Proceedings of the 13th International Joint Conference on Artificial Intelligence, 1022–1027. https://ijcai.org/proceedings/1993/141

6. Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. Proceedings of the 14th International Joint Conference on Artificial Intelligence, 1137–1144. https://ijcai.org/proceedings/1995/147

7. Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. Information Processing & Management, 45(4), 427–437. https://doi.org/10.1016/j.ipm.2009.03.002

8. Cortez, P. (2008). Student Performance [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5TG7T

9. Cortez, P., & Silva, A. M. G. (2008). Using data mining to predict secondary school student performance. Proceedings of the 5th Future Business Technology Conference.

10. Baker, R. S., & Inventado, P. S. (2014). Educational data mining and learning analytics. In J. A. Larusson & B. White (Eds.), Learning Analytics: From Research to Practice (pp. 61–75). Springer. https://doi.org/10.1007/978-1-4614-3305-7_4

11. Han, J., Kamber, M., & Pei, J. (2012). Data Mining: Concepts and Techniques (3rd ed.). Morgan Kaufmann.

12. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). An Introduction to Statistical Learning (2nd ed.). Springer. https://www.statlearning.com/

13. Domingos, P. (2012). A few useful things to know about machine learning. Communications of the ACM, 55(10), 78–87. https://doi.org/10.1145/2347736.2347755

14. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

15. Scikit-learn Developers. (n.d.). Decision trees: User guide. https://scikit-learn.org/stable/modules/tree.html